

Mecanismo de Penalización para Procesos CPU-bound

Informe Técnico — Modificación del Scheduler de MINIX 3

THE MATCOM MINIX — Sistemas Operativos

Miguel Zamora Grupo: C-212
Alfonso Tejeda Rodríguez Grupo: C-212

14 de mayo de 2026

Índice

1. Introducción	2
2. Arquitectura del scheduler en MINIX	2
2.1. El scheduler como servidor de espacio de usuario	2
2.2. Funciones principales de <code>schedule.c</code>	2
2.3. Convención de prioridades	3
3. Problema: limitaciones del scheduler original	3
3.1. Comportamiento antes de la modificación	3
3.2. Deficiencias identificadas	4
4. Diseño de la solución	4
4.1. Idea central: observación por ventanas temporales	4
4.2. Parámetros de la política y justificación	4
4.3. Nuevos campos de estado por proceso	5
5. Implementación	5
5.1. Cambio 1 — <code>schedproc.h</code> : nuevos campos en la estructura	5
5.2. Cambio 2 — <code>schedule.c</code> : constantes de configuración	5
5.3. Cambio 3 — <code>do_noquantum()</code> : punto de observación	6
5.4. Cambio 4 — <code>do_start_scheduling()</code> : inicialización	6
5.5. Cambio 5 — <code>balance_queues()</code> : corazón de la política	7
6. Tabla de decisiones de <code>balance_queues</code>	9
7. Flujo completo del mecanismo	9
7.1. Ejemplo numérico de evolución de prioridad	10
8. Validación experimental	10
8.1. Programas de prueba	10
8.2. Procedimiento	10
8.3. Resultados observados	11
9. Resumen de cambios	11
10. Conclusiones	12

1. Introducción

Este documento describe el diseño, implementación y validación del mecanismo de penalización para procesos *CPU-bound* introducido en el planificador de espacio de usuario de MINIX 3. El objetivo es que el scheduler distinga entre procesos que consumen CPU de forma sostenida y procesos interactivos que ceden la CPU voluntariamente, aplicando una penalización de prioridad gradual a los primeros y permitiendo su recuperación cuando cambian de comportamiento.

El trabajo modifica dos archivos del servidor `sched`:

- `minix/servers/sched/schedproc.h` — estructura de datos por proceso.
- `minix/servers/sched/schedule.c` — lógica del planificador.

2. Arquitectura del scheduler en MINIX

2.1. El scheduler como servidor de espacio de usuario

MINIX 3 implementa un **planificador en espacio de usuario**. A diferencia de los kernels monolíticos donde la política de planificación vive dentro del kernel, en MINIX el kernel solo provee mecanismos básicos (colas de ejecución, interrupciones de reloj, context switch) y delega todas las decisiones de planificación al servidor `sched`, que corre como un proceso de sistema independiente.

La interacción funciona mediante paso de mensajes:

1. El kernel detecta un evento relevante (proceso agotó su quantum, proceso nuevo, proceso terminado).
2. El kernel envía un mensaje IPC al servidor `sched`.
3. El servidor `sched` actualiza su estado interno y responde indicando al kernel qué hacer (qué prioridad y quantum asignar al proceso).
4. El kernel aplica la decisión.

Esta separación tiene consecuencias prácticas importantes: un error en el scheduler no provoca un kernel panic; la política puede modificarse y reinstalarse sin recompilar el kernel completo; y es posible incluso tener múltiples schedulers activos para distintos conjuntos de procesos.

2.2. Funciones principales de `schedule.c`

Función	Rol
<code>do_noquantum()</code>	Invocada cuando el kernel detecta que un proceso agotó su quantum sin bloquearse. Punto de observación del comportamiento CPU-intensivo.
<code>do_start_scheduling()</code>	Registra un proceso nuevo en el scheduler. Maneja dos variantes: <code>SCHEDULING_START</code> (proceso de sistema) y <code>SCHEDULING_INHERIT</code> (proceso de usuario que hereda del padre).
<code>do_stop_scheduling()</code>	Libera el slot cuando el proceso termina o es detenido.
<code>do_nice()</code>	Atiende la syscall <code>nice</code> : modifica la prioridad máxima permitida.

Función	Rol
balance_queues()	Función de balanceo periódico, disparada cada <code>BALANCE_TIMEOUT = 5</code> s. Era el único mecanismo de recuperación de prioridad en el scheduler original.
schedule_process()	Función interna que comunica al kernel los cambios de prioridad, quantum y CPU mediante <code>sys_schedule()</code> .

2.3. Convención de prioridades

Las prioridades en MINIX están representadas como enteros con semántica **invertida**: un número *menor* corresponde a *mayor* prioridad. La cola 0 es la más prioritaria y está reservada al kernel; los procesos de usuario operan en el rango `USER_Q-MIN_USER_Q`:

Constante	Valor típico	Significado
<code>USER_Q</code>	7	Prioridad base normal de proceso de usuario
<code>MIN_USER_Q</code>	14	Peor prioridad permitida (número más alto del rango)
<code>max_priority</code>	variable	Prioridad máxima permitida para el proceso

Consecuencia directa: penalizar un proceso significa *aumentar* su número de cola; recuperarlo significa *disminuirlo*. Esta inversión es crítica para entender todos los cálculos del mecanismo implementado.

3. Problema: limitaciones del scheduler original

3.1. Comportamiento antes de la modificación

El scheduler original tenía la siguiente lógica en sus dos funciones críticas.

`do_noquantum()` original:

Listing 1: `do_noquantum()` antes de la modificación

```

1 rmp = &schedproc[proc_nr_n];
2 if (rmp->priority < MIN_USER_Q) {
3     rmp->priority += 1; /* lower priority */
4 }
```

`balance_queues()` original:

Listing 2: `balance_queues()` antes de la modificación

```

1 for (proc_nr = 0, rmp = schedproc;
2     proc_nr < NR_PROCS; proc_nr++, rmp++) {
3     if (rmp->flags & IN_USE) {
4         if (rmp->priority > rmp->max_priority) {
5             rmp->priority -= 1; /* increase priority */
6             schedule_process_local(rmp);
7         }
8     }
9 }
```

3.2. Deficiencias identificadas

1. **Sin memoria del comportamiento pasado.** El scheduler no almacenaba información histórica. Cada decisión se tomaba sin contexto sobre cómo se había comportado el proceso en el pasado.
2. **Trato idéntico para todos los perfiles de ejecución.** Un proceso que agotara su quantum 20 veces por segundo recibía exactamente el mismo tratamiento que uno que cediera la CPU voluntariamente en cada ciclo.
3. **Sin distinción entre procesos de usuario y de sistema.** `balance_queues()` aplicaba su lógica de recuperación a todos los procesos sin filtrar, mezclando servicios del sistema con procesos de usuario.
4. **Recuperación siempre completa en una sola ventana.** Un proceso CPU-bound degradado varios niveles podía recuperar su prioridad en la siguiente ventana simplemente por el paso del tiempo, sin importar si seguía siendo intensivo en CPU.

4. Diseño de la solución

4.1. Idea central: observación por ventanas temporales

La política implementada introduce una **ventana de observación de 5 segundos**. Durante cada ventana el scheduler cuenta cuántas veces cada proceso agota su quantum completo. Al final de la ventana clasifica el proceso según ese conteo:

- Agotó muchos quantums $\Rightarrow$ CPU-bound $\Rightarrow$ penalizar (bajar prioridad).
- No agotó ninguno y tenía penalización $\Rightarrow$ recuperar gradualmente.
- Agotó pocos (zona neutral) $\Rightarrow$ no se toma acción.

4.2. Parámetros de la política y justificación

Ventana temporal: 5 segundos. Se reutiliza el temporizador `BALANCE_TIMEOUT = 5` que ya disparaba `balance_queues()`. Esto evita introducir una segunda alarma del sistema y mantiene la nueva política sincronizada con el balanceo original en un único punto de control.

Umbral CPU-bound: `CPU_BOUND_QUANTUM_THRESHOLD = 3`. Un proceso que agote 3 o más quantums completos en 5 segundos se clasifica como CPU-intensivo. El valor 3 es lo suficientemente bajo para detectar comportamiento sostenido pero lo suficientemente alto para ignorar ráfagas breves de cómputo.

Penalización máxima: `MAX_PENALTY_LEVEL = 2`. La prioridad puede bajar hasta 2 niveles por debajo de la base. Este techo evita inanición y garantiza que la recuperación sea viable en 2 ventanas de 5 segundos.

Recuperación gradual: un nivel por ventana. Si el proceso no agota ningún quantum y tiene penalización activa, recupera un nivel. La recuperación es simétrica a la penalización para que un instante de inactividad forzada no borre todo el historial de comportamiento CPU-intensivo.

Zona neutral: `quantum_count` de 1 a 2. Cuando el conteo está por debajo del umbral pero por encima de cero no se toma ninguna acción. Cierta actividad de CPU es normal y no merece ni castigo ni recuperación especial.

4.3. Nuevos campos de estado por proceso

Para implementar la política, cada proceso necesita tres datos adicionales:

Campo	Tipo	Rol
base_priority	unsigned	Prioridad de referencia. La prioridad efectiva es siempre <code>base_priority + penalty_level</code> . Sin este campo no habría forma de deshacer el castigo ni de saber cuánto castigo acumulado tiene el proceso.
quantum_count	unsigned	Quantums completos agotados en la ventana actual. Se incrementa en <code>do_noquantum()</code> y se reinicia a 0 en cada llamada a <code>balance_queues()</code> . Es la métrica que distingue CPU-bound de interactivo.
penalty_level	unsigned	Nivel de castigo acumulado (0 = sin castigo, máx. <code>MAX_PENALTY_LEVEL</code> = 2). La prioridad efectiva es <code>base_priority + penalty_level</code> .

5. Implementación

5.1. Cambio 1 — schedproc.h: nuevos campos en la estructura

Listing 3: Diff de schedproc.h

```
1  EXTERN struct schedproc {
2      endpoint_t endpoint;
3      endpoint_t parent;
4      unsigned flags;
5      unsigned max_priority;
6      unsigned priority;
7      unsigned time_slice;
8      unsigned cpu;
9      bitchunk_t cpu_mask[BITMAP_CHUNKS(CONFIG_MAX_CPUS)];
10 +
11 +
12 + /* CPU-bound process penalty mechanism */
13 + unsigned base_priority; /* base priority for recovery */
14 + unsigned quantum_count; /* quanta consumed this window */
15 + unsigned penalty_level; /* current penalty (0 = none) */
16 } schedproc[NR_PROCS];
```

Los tres campos se agregan al final de la estructura para no alterar la disposición de memoria de los campos existentes. Quedan agrupados bajo un comentario que los identifica como parte del mismo mecanismo.

5.2. Cambio 2 — schedule.c: constantes de configuración

Listing 4: Constantes de la política

```
1 /* CPU-bound process penalty mechanism */
2 #define CPU_BOUND_QUANTUM_THRESHOLD 3 /* umbral de quanta */
3 #define MAX_PENALTY_LEVEL          2 /* castigo maximo */
```

Definirlas como macros con nombre explícito permite ajustar toda la política modificando solo estas dos líneas, sin tocar la lógica de las funciones.

5.3. Cambio 3 — do_noquantum(): punto de observación

Esta función se invoca cada vez que el kernel detecta que un proceso agotó su quantum sin ceder la CPU. Es el único punto donde se sabe con certeza que el proceso ejecutó de forma continua hasta ser interrumpido por el reloj.

Listing 5: Diff de do_noquantum()

```
1  rmp = &schedproc[proc_nr_n];
2
3  /* Count full quantum consumed by this process */
4  +if (!is_system_proc(rmp)) {
5  +    rmp->quantum_count++;
6  +}
7  +
8  if (rmp->priority < MIN_USER_Q) {
9      rmp->priority += 1; /* lower priority */
10 }
```

Por qué la guarda `!is_system_proc(rmp)`. La macro comprueba si el padre del proceso es el Reincarnation Server (`RS_PROC_NR`), que lanza y supervisa todos los servicios del sistema (VFS, PM, drivers, etc.). Los procesos de sistema pueden agotar quanta por razones estructurales (atender ráfagas de peticiones IPC) que no reflejan comportamiento abusivo. Penalizarlos podría degradar la respuesta del sistema. La exclusión es conservadora y correcta.

Por qué no se modifica la lógica original. El bloque `priority += 1` existente se conserva intacto; el contador se añade *antes* sin interferir. La degradación inmediata dentro de la ventana sigue funcionando como siempre, y la nueva información se usará al final de la ventana en `balance_queues()`.

5.4. Cambio 4 — do_start_scheduling(): inicialización

Cuando el scheduler registra un proceso nuevo, los campos del slot de `schedproc[]` pueden contener basura del proceso anterior. Los tres campos nuevos se inicializan explícitamente en tres rutas de código distintas.

Bloque temprano (antes del switch). Establece valores seguros para todos los procesos — incluido el caso especial de `init` — y garantiza que ningún proceso nuevo herede estado de penalización corrupto del slot anterior.

Listing 6: Inicialización temprana en do_start_scheduling()

```
1  /* ... validar max_priority ... */
2  if (rmp->max_priority >= NR_SCHED_QUEUES) {
3      return EINVAL;
4  }
5
6  /* Initialize penalty mechanism fields */
7  +rmp->base_priority = USER_Q;
8  +rmp->quantum_count = 0;
9  +rmp->penalty_level = 0;
```

Rama `SCHEDULING_START` (procesos de sistema). `base_priority` debe reflejar la prioridad real asignada (`max_priority`), no el valor genérico `USER_Q`. Si se usara `USER_Q = 7` como base para un proceso con prioridad 4, los cálculos producirían resultados sin sentido.

Listing 7: Rama SCHEDULING_START

```

1  case SCHEDULING_START:
2      rmp->priority      = rmp->max_priority;
3  +   rmp->base_priority = rmp->max_priority;
4      rmp->time_slice    = /* ... quantum del mensaje ... */;
5  +   rmp->quantum_count = 0;
6  +   rmp->penalty_level = 0;
7      break;

```

Rama SCHEDULING_INHERIT (procesos de usuario). El hijo hereda la prioridad *actual* del padre como `base_priority`. Se usa `priority` (no `base_priority`) del padre porque es la prioridad real con la que el hijo comienza a ejecutarse. El hijo no hereda el historial del padre: parte con `quantum_count = 0` y `penalty_level = 0`, construyendo su reputación de forma independiente.

Listing 8: Rama SCHEDULING_INHERIT

```

1  case SCHEDULING_INHERIT:
2      rmp->priority = schedproc[parent_nr_n].priority;
3  +   rmp->base_priority = schedproc[parent_nr_n].priority;
4      rmp->time_slice = schedproc[parent_nr_n].time_slice;
5  +   rmp->quantum_count = 0;
6  +   rmp->penalty_level = 0;
7      break;

```

5.5. Cambio 5 — `balance_queues()`: corazón de la política

Esta es la función más significativamente modificada. Se ejecuta cada 5 segundos y aplica la política completa de penalización y recuperación.

Listing 9: Nueva implementación de `balance_queues()`

```

1  void balance_queues(void)
2  {
3      struct schedproc *rmp;
4      int r, proc_nr;
5      unsigned new_priority; /* nueva variable local */
6
7      for (proc_nr = 0, rmp = schedproc;
8           proc_nr < NR_PROCS;
9           proc_nr++, rmp++) {
10
11         if (rmp->flags & IN_USE && !is_system_proc(rmp)) {
12             /* Rama: procesos de usuario */
13
14             if (rmp->quantum_count >= CPU_BOUND_QUANTUM_THRESHOLD) {
15                 /* CPU-bound: incrementar penalizacion */
16                 if (rmp->penalty_level < MAX_PENALTY_LEVEL)
17                     rmp->penalty_level++;
18                 new_priority =
19                     rmp->base_priority + rmp->penalty_level;
20                 if (new_priority < MIN_USER_Q)
21                     new_priority = MIN_USER_Q; /* clamp inferior */
22                 if (rmp->priority != new_priority) {
23                     rmp->priority = new_priority;
24                     schedule_process_local(rmp);
25                 }
26
27             } else if (rmp->quantum_count == 0

```

```

28         && rmp->penalty_level > 0) {
29             /* Inactivo con penalizacion: recuperar un nivel */
30             rmp->penalty_level--;
31             new_priority =
32                 rmp->base_priority + rmp->penalty_level;
33             if (new_priority > rmp->max_priority)
34                 new_priority = rmp->max_priority; /* clamp */
35             if (rmp->priority != new_priority) {
36                 rmp->priority = new_priority;
37                 schedule_process_local(rmp);
38             }
39         }
40
41         rmp->quantum_count = 0; /* reiniciar ventana */
42
43     } else if (rmp->flags & IN_USE) {
44         /* Procesos de sistema: comportamiento original */
45         if (rmp->priority > rmp->max_priority) {
46             rmp->priority -= 1;
47             schedule_process_local(rmp);
48         }
49     }
50 }
51
52 if ((r = sys_setalarm(balance_timeout, 0)) != OK)
53     panic("sys_setalarm failed: %d", r);
54 }

```

Decisiones de implementación

Clamp inferior en penalización. La condición es `new_priority < MIN_USER_Q`. Puede parecer contraintuitiva (se esperaría `>`), pero recuerde la semántica invertida: en MINIX, `MIN_USER_Q` es el número *más alto* del rango de usuario. Si `base_priority + penalty_level` superara ese valor, el proceso entraría en el rango de colas de sistema con consecuencias imprevisibles. El clamp garantiza que ningún proceso de usuario salga de su rango válido.

Clamp superior en recuperación. La condición es `new_priority > rmp->max_priority`. Al recuperar prioridad (disminuir el número de cola) hay que asegurarse de no superar la prioridad máxima autorizada del proceso. Un proceso con `nice` aplicado no debería recuperarse hasta `USER_Q` si ya no tiene ese permiso.

Condición `rmp->priority != new_priority`. Solo se llama a `schedule_process_local()` si la prioridad efectivamente cambia. Esto evita tráfico IPC innecesario al kernel. Por ejemplo, un proceso en el techo de penalización que siga siendo CPU-bound ya tiene `priority == MIN_USER_Q`: se calculará `new_priority == MIN_USER_Q`, la condición fallará y no se enviará ningún mensaje.

Reinicio de `quantum_count` fuera del `if/else if`. El reinicio ocurre *siempre* al final del bloque, sin importar cuál fue el resultado de la evaluación. Si estuviera dentro de alguna rama, los procesos en zona neutral (`quantum_count` de 1 o 2) no reiniciarían su contador entre ventanas, acumulando conteos y corrompiendo la semántica de “quantums agotados en la ventana *actual*”.

Separación explícita sistema / usuario. La condición `!is_system_proc(rmp)` separa completamente los dos tipos. Los procesos de sistema caen en el `else if`, donde se conserva exac-

tamente el comportamiento original: si fueron degradados, suben un nivel. Esta separación es explícita y sin ambigüedad: no hay riesgo de regresiones en la estabilidad del sistema.

6. Tabla de decisiones de balance_queues

La siguiente tabla resume todas las decisiones posibles para cada combinación de estado de proceso:

Tipo	quantum_count	penalty_level	Acción
Usuario	≥ 3	< 2	<code>penalty_level++</code> ; <code>priority = base + penalty</code> ; clamp a <code>MIN_USER_Q</code> ; reschedule si cambió
Usuario	≥ 3	$= 2$ (techo)	Prioridad se mantiene en techo; no se reschedule
Usuario	1 o 2	cualquiera	Sin cambio (zona neutral)
Usuario	$= 0$	> 0	<code>penalty_level--</code> ; <code>priority = base + penalty</code> ; clamp a <code>max_priority</code> ; reschedule si cambió
Usuario	$= 0$	$= 0$	Sin cambio (ya en prioridad base)
Sistema	—	—	Comportamiento original: sube 1 nivel si fue degradado
En todos los casos de usuario: <code>quantum_count = 0</code> al final de la ventana.			

7. Flujo completo del mecanismo

El ciclo de vida de la penalización para un proceso CPU-bound sigue estos pasos:

1. **Creación.** `do_start_scheduling()` inicializa: `priority = base_priority = USER_Q = 7`, `quantum_count = 0`, `penalty_level = 0`.
2. **Ejecución intensa.** El proceso ejecuta código CPU-intensivo. Cada vez que agota su quantum, el kernel envía `SCHEDULING_NO_QUANTUM`. `do_noquantum()` incrementa `quantum_count` y baja un nivel la prioridad inmediata (`priority += 1`), según el comportamiento original conservado.
3. **Fin de ventana — primer castigo.** `balance_queues()` se ejecuta. Como `quantum_count >= 3`: `penalty_level` sube a 1, `priority` se fija en `base + 1 = 8`. `quantum_count` se reinicia a 0.
4. **Segunda ventana — castigo máximo.** El proceso sigue siendo CPU-bound. `penalty_level` sube a 2, `priority` se fija en 9. Ya está en el techo.
5. **Ventanas subsiguientes — techo estable.** `penalty_level` no puede superar `MAX_PENALTY_LEVEL = 2`. La prioridad permanece en 9 mientras el proceso siga agotando ≥ 3 quants por ventana.
6. **Cambio de comportamiento — inicio de recuperación.** El proceso empieza a hacer I/O y no agota ningún quantum en una ventana. `quantum_count == 0` y `penalty_level > 0`: `penalty_level` baja a 1, `priority` vuelve a 8.
7. **Recuperación completa.** Una ventana más sin agotar quants: `penalty_level` baja a 0, `priority` vuelve a 7. El proceso queda totalmente recuperado.

7.1. Ejemplo numérico de evolución de prioridad

Valores: `base_priority = USER_Q = 7`, `MIN_USER_Q = 14`, `MAX_PENALTY_LEVEL = 2`.

Ventana	q_count	penalty	priority	Descripción
0	—	0	7	Proceso recién creado
1	5	1	8	CPU-bound → primer castigo
2	4	2	9	CPU-bound → castigo máximo (techo)
3	6	2	9	Sigue CPU-bound, techo ya alcanzado
4	0	1	8	Interactivo → recupera 1 nivel
5	0	0	7	Totalmente recuperado

8. Validación experimental

8.1. Programas de prueba

Proceso CPU-bound — bucle aritmético infinito que nunca cede la CPU:

Listing 10: `cpu_bound.c`

```
1 int main(void) {
2     volatile long x = 0;
3     while (1) x++; /* nunca cede la CPU voluntariamente */
4     return 0;
5 }
```

La variable `volatile` impide que el compilador elimine el bucle como código muerto.

Proceso interactivo — proceso que cede la CPU en cada iteración mediante `sleep`:

Listing 11: `interactive.c`

```
1 #include <unistd.h>
2 #include <stdio.h>
3 int main(void) {
4     while (1) {
5         sleep(1); /* bloqueo voluntario, no agota quantum */
6         printf("tick\n");
7     }
8     return 0;
9 }
```

Un proceso bloqueado en `sleep` libera la CPU; el kernel nunca envía `SCHEDULING_NO_QUANTUM` por él, de modo que `quantum_count` permanece en 0 durante toda su ejecución.

8.2. Procedimiento

```
cc -o cpu_bound cpu_bound.c
cc -o interactive interactive.c
./cpu_bound &
./interactive &
top
```

8.3. Resultados observados

Ventana	CPU-bound	Interactivo	Observación
0–5 s	7 (base)	7 (base)	Ambos arrancan con <code>USER_Q</code>
5–10 s	8 (penalizado 1)	7 (sin cambio)	CPU-bound agotó ≥ 3 quantums
10–15 s	9 (penalizado 2)	7 (sin cambio)	Alcanza techo de penalización
15–20 s	9 (estable)	7 (sin cambio)	Penalización en techo máximo
Al detener el proceso CPU-bound (pasa a no agotar quantums):			
20–25 s	9 $\rightarrow$ 8	7 (estable)	<code>q_count = 0</code> : recupera 1 nivel
25–30 s	8 $\rightarrow$ 7	7 (estable)	Recuperación completa en 2 ventanas

Los resultados confirman que:

- El proceso CPU-bound fue penalizado progresivamente, un nivel por ventana, hasta el techo definido por `MAX_PENALTY_LEVEL`.
- El proceso interactivo no fue afectado en ningún momento.
- La recuperación fue gradual y simétrica a la penalización.
- Los límites `MIN_USER_Q` y `max_priority` se respetaron en todo momento.

9. Resumen de cambios

Elemento	Tipo	Descripción
<code>schedproc.h: base_priority</code>	Campo nuevo	Prioridad de referencia para calcular penalización y recuperación
<code>schedproc.h: quantum_count</code>	Campo nuevo	Contador de quantums agotados en la ventana actual
<code>schedproc.h: penalty_level</code>	Campo nuevo	Nivel de castigo acumulado (0–2)
<code>schedule.c: CPU_BOUND_QUANTUM_THRESHOLD</code>	Constante nueva	Umbral de quantums para activar castigo (valor: 3)
<code>schedule.c: MAX_PENALTY_LEVEL</code>	Constante nueva	Castigo máximo acumulable (valor: 2)
<code>schedule.c: do_noquantum()</code>	Modificada	Incrementa <code>quantum_count</code> en procesos de usuario
<code>schedule.c: do_start_scheduling()</code>	Modificada	Inicializa los 3 nuevos campos en 3 puntos de la función
<code>schedule.c: balance_queues()</code>	Modificada	Implementa la política completa de penalización y recuperación

10. Conclusiones

La modificación introduce en el scheduler de MINIX una política basada en ventanas temporales que distingue entre procesos CPU-bound e interactivos. Las decisiones de diseño clave fueron:

- **Reutilizar** `BALANCE_TIMEOUT` como ventana de observación, evitando la complejidad de una segunda alarma del sistema.
- **Mantener memoria por proceso** mediante tres campos nuevos (`base_priority`, `quantum_count`, `penalty_level`), que permiten penalización y recuperación graduales sin perder el punto de referencia.
- **Excluir procesos de sistema** del mecanismo de conteo, preservando la estabilidad del sistema y respetando la diferencia estructural entre servicios del kernel y procesos de usuario.
- **Parametrizar la política** mediante las macros `CPU_BOUND_QUANTUM_THRESHOLD` y `MAX_PENALTY_LEVEL`, lo que permite ajustar la agresividad sin modificar la lógica del scheduler.
- **Conservar el comportamiento original** para procesos de sistema en `balance_queues()`, garantizando que la modificación no introduce regresiones en el resto del sistema.

Como posibles extensiones se identifican: (1) exponer `quantum_count` y `penalty_level` por proceso mediante `sysctl` o similar, para facilitar la validación sin depender de `top`; (2) introducir un umbral de recuperación más suave que permita recuperar prioridad cuando `quantum_count` decrece entre ventanas pero no llega a cero; (3) parametrizar la duración de la ventana de forma independiente al `BALANCE_TIMEOUT` para ajustar ambas políticas de forma ortogonal.